

Main Lorentz

Main–Lorentz 算法

重串

定义

给定一个长度为 n 的字符串 S 。

我们将一个字符串连续写两遍所产生的新字符串称为 **重串 (tandem repetition)**。下文中，为了表述精准，我们将被重复的这个字符串称为原串。换言之，一个重串等价于一对下标 (i, j) ，其使得 $S[i:j]$ 是两个相同字符串拼接而成的。

你的目标是找出给定的字符串 S 中所有的重串。或者，解决一个较为简单的问题：找到字符串 S 中任意重串或者最长的一个重串。

下文的算法由 Michael Main 和 Richard J. Lorentz 在 1982 年提出。

声明：

下文所有的字符串下标从 0 开始。

下文中，记 $S[i:j]$ 为 S 的反串。如 $S[i:j] = S[j:i]$ 。

解释

考虑字符串 $S = \text{abcabcabc}$ ，这个字符串包括三个重串，分别是：

- $S[0:3]$
- $S[3:6]$
- $S[6:9]$

下面是另一个例子，考虑字符串 $S = \text{ababab}$ ，这个字符串只有两个重串：

- $S[0:2]$
- $S[2:4]$

重串的个数

一个长度为 n 的字符串可能有高达 n 个重串，一个显然的例子就是 n 个字母全部相同的字符串，这种情况下，只要其子串长度为偶数，这个子串就是重串。多数情况下，一个周期比较小的周期字符串会有很多重串。

但这并不影响我们在 $O(n)$ 的时间内计算出重串数量。因为这个算法通过某种压缩形式来表达一个重串，使得我们可以将多个重串压缩为一个。

这里有一些关于重串数量的有趣结论：

- 如果一个重串的原串不是重串，则我们称这个重串为 **本原重串 (primitive repetition)**。可以证明，本原重串最多有 $\sqrt{n}$ 个。
- 如果我们把一个重串用 Crochemore 三元组 (i, p, q) 进行压缩，其中 i 是重串的起始位置， p 是该重串某个循环节的长度（注意不是原串长度！）， q 为这个循环节重复的次数。则某字符串的所有重串可以被 $O(\sqrt{n})$ 个 Crochemore 三元组表示。
- Fibonacci 字符串定义如下：

可以发现 Fibonacci 字符串具有高度的周期性。对于长度为 n 的 Fibonacci 字符串，即使用 Crochemore 三元组压缩，也

有一个三元组。其本原重串的数量也有一个。

Main–Lorentz 算法

解释

Main–Lorentz 算法的核心思想是 **分治**。

这个算法将字符串分为左、右两部分，首先计算完全处于字符串左部（或右部）的重串数量，然后计算起始位置在左部，终止在右部的重串数量。（下文中，我们将这种重串称为 **交叉重串**）

计算交叉重串的数量是 Main–Lorentz 算法的关键点，我们将在下文详细探讨。

过程

寻找交叉重串

我们记某字符串的左部为 L ，右部为 R 。则 $|L|$ 且 $|R|$ 的长度大约等于 $|S|$ 长度的一半。

对于任意一个重串，我们考虑其中间字符。此处我们将一个重串右半边的第一个字符称为其中间字符，换言之，若 $S[i..j]$ 为重串，则其中间字符为 $S[\lfloor \frac{i+j}{2} \rfloor]$ 。如果一个重串的中间字符在 L 中，则称这个重串 **左偏 (left)**，反之则称其 **右偏 (right)**。

接下来，我们将会探讨如何找到所有的左偏重串。

我们记一个左边重串的长度为 l 。考虑该重串第一个落入 R 的字符（即 $S[i]$ ），这个字符一定与 L 中的某个字符一致。

我们考虑固定 i ，并找到所有符合条件的重串。举个例子：对于字符串 $S = \text{abababab}$ （这个 S 是用于分辨左右两部分的），固定 $i = 4$ ，则我们可以发现重串 $S[2..6]$ 符合要求。

显然，我们一旦固定了 i ，那我们同时也固定了 l 的取值。我们一旦知道如何找到所有重串，我们就可以从 1 到 $|S|$ 枚举 l 的取值，然后找到所有符合条件的重串。

左偏重串的判定

即使固定 i 后，仍然可能会有多个符合条件的重串，我们怎么找到所有符合条件的重串呢？

我们再来举一个例子，对于字符串 S 中的重串 $S[i..j]$ ，我们记 l 为该重串的首字符到 $S[i]$ 所组成的子串的长度，记 r 为 $S[i]$ 到该重串原串的末字符所组成的子串的长度。

于是，我们可以给出某个长度为 l 的子串是重串的 **充分必要条件**：

记 l 为满足 $l \leq |L|$ 的最大整数，记 r 为满足 $r \leq |R|$ 的最大整数。则对于任意满足 $l + r = |S|$ 的二元组 (l, r) ，我们都能恰好找到一个与之对应的重串。

总结一下，即有：

- 固定一个 i 。
- 那么我们此时要找的重串长度均为 $l + r$ 。此时可能仍有多个符合条件的重串，取决于 l 与 r 的取值。
- 计算上文提到的 l 和 r 。
- 则所有符合条件的重串符合条件：

接下来，只需要考虑如何快速算出 l 与 r 了。借助 [Z 函数](#)，我们可以计算它们：

- 计算 l ：只需计算 S 的 Z 函数即可。
- 计算 r ：只需计算 S 的 Z 函数即可，其中 S' 是一个 $|S|$ 中均没有的字符。

右偏重串

计算右偏重串的方法与计算左偏重串的方法几乎一致。考虑该重串第一个落入 的字符（即），则其一定与 中的某个字符一致，记这个字符在 中的位置为。

令 为满足 的最大整数， 为满足 的最大整数。则我们可以分别通过计算 和 的 Z 函数来得出 与。

枚举，用相仿的方法寻找右偏重串即可。

实现

Main–Lorentz 算法以四元组 的形式给出所有重串。如果你只需要计算重串的数量，或者只需要找到最长的一个重串，这个四元组给的信息是足够的。由 [主定理](#) 可得，Main–Lorentz 算法的时间复杂度为。

请注意，如果你想通过这些四元组来找到所有重串的起始位置与终止位置，则最坏时间复杂度会达到。我们在下面的程序中实现了这一点，将所有重串的起始位置与终止位置存于 repetitions 中。

```
1 vector<int> z_function(string const& s) {
2     int n = s.size();
3     vector<int> z(n);
4     for (int i = 1, l = 0, r = 0; i < n; i++) {
5         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
6         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
7         if (i + z[i] - 1 > r) {
8             l = i;
9             r = i + z[i] - 1;
10        }
11    }
12    return z;
13 }
14
15 int get_z(vector<int> const& z, int i) {
16     if (0 <= i && i < (int)z.size())
17         return z[i];
18     else
19         return 0;
20 }
21
22 vector<pair<int, int>> repetitions;
23
24 void convert_to_repetitions(int shift, bool left, int cntr, int l, int k1,
25                             int k2) {
26     for (int l1 = max(l, l - k2); l1 <= min(l, k1); l1++) {
27         if (left && l1 == l) break;
28         int l2 = l - l1;
29         int pos = shift + (left ? cntr - l1 : cntr - l - l1 + 1);
30         repetitions.emplace_back(pos, pos + 2 * l - 1);
31     }
32 }
33
34 void find_repetitions(string s, int shift = 0) {
35     int n = s.size();
36     if (n == 1) return;
37
38     int nu = n / 2;
39     int nv = n - nu;
40     string u = s.substr(0, nu);
41     string v = s.substr(nu);
42     string ru(u.rbegin(), u.rend());
43     string rv(v.rbegin(), v.rend());
44
45     find_repetitions(u, shift);
46     find_repetitions(v, shift + nu);
47
48     vector<int> z1 = z_function(ru);
49     vector<int> z2 = z_function(v + '#' + u);
```

```
50 vector<int> z3 = z_function(ru + '#' + rv);
51 vector<int> z4 = z_function(v);
52
53 for (int cntr = 0; cntr < n; cntr++) {
54     int l, k1, k2;
55     if (cntr < nu) {
56         l = nu - cntr;
57         k1 = get_z(z1, nu - cntr);
58         k2 = get_z(z2, nv + 1 + cntr);
59     } else {
60         l = cntr - nu + 1;
61         k1 = get_z(z3, nu + 1 + nv - 1 - (cntr - nu));
62         k2 = get_z(z4, (cntr - nu) + 1);
63     }
64     if (k1 + k2 >= l) convert_to_repetitions(shift, cntr < nu, cntr, l, k1, k2);
65 }
66 }
```

本页面主要译自博文 [Поиск всех тандемных повторов в строке. Алгоритм Мейна-Лоренца](#) 与其英文翻译版 [Finding repetitions](#)。其中俄文版版权协议为 **Public Domain + Leave a Link**；英文版版权协议为 **CC-BY-SA 4.0**。

本页面最近更新：2023/6/27 13:39:08, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [billchenchina](#), [Enter-tainer](#), [fsy-juruo](#), [iamtwz](#), [ouuan](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用